

DRL Portfolio — architecture technique

Comment c'est construit : pipeline, environnement, entraînement, générateur de scénarios, tests

(complément d'implémentation au rapport — qui, lui, porte le raisonnement financier)

Martin Chassaing

État du code au 5 juillet 2026 — document instantané, non maintenu en continu

1 Vue d'ensemble : deux sous-systèmes, un juge commun

Le dépôt contient deux sous-systèmes indépendants qui partagent la même discipline d'évaluation :

1. **L'agent de trading** : `data_loader.py` → `environment.py` → `train.py` (PPO/SB3) → `evaluate.py` / `walk_forward.py` / `seed_robustness.py`.
2. **Le générateur de scénarios** (volet IA générative) : `package diffusion/` → `train_diffusion.py` → `validate_diffusion.py` (protocole pré-enregistré É1–É6).

Le principe transversal : *chaque étage est testé sur des données synthétiques aux propriétés connues avant d'être branché au suivant* (103 tests pytest, dont aucun ne dépend du réseau hors marqueurs dédiés). Les interfaces entre étages sont des artefacts explicites sur disque : `models/<exp>/` (poids + normalisation adjacente), `reports/*.json` (métriques), `reports/figures/*.png`.

2 Pipeline de données (`data_loader.py`)

Source de vérité unique : la dataclass `DataConfig` porte dates (2010–2023), tickers (AAPL, MSFT, GOOGL, SPY, TSLA) et ratios de split (70/15/15). Aucun script ne re-hardcode de dates.

Chaîne : `_download` (yfinance, `Close` ajusté uniquement) → `_build_features` → normalisation → split chronologique strict. Les 5 features et leur définition exacte :

<code>log_returns</code>	$\ln(P_t/P_{t-1})$	stationnarise le prix
<code>volatility</code>	std glissante 20 j des log-rendements	proxy GARCH
<code>rsi</code>	$\text{RSI}(14)/100$	borné $[0, 1]$
<code>macd_norm</code>	$(\text{EMA}_{12} - \text{EMA}_{26})/\text{EMA}_{26}$	momentum directionnel
<code>momentum_5</code>	$P_t/P_{t-5} - 1$	signal court terme

Anti look-ahead : le `RobustScaler` est *fitté sur le train uniquement* puis appliqué à val/test — la ligne la plus importante du fichier. `FEATURES` est volontairement dupliqué dans `environment.py` et un test vérifie la synchronisation des deux listes.

Multi-ticker : chaque ticker est splitté *séparément* (70/15/15 sur sa propre histoire), puis les splits sont concaténés avec une colonne `segment_id`. C'est elle qui permet à l'environnement de confiner les épisodes à un seul actif.

Piège vécu. Sans `segment_id` sur les trois splits, un épisode de validation pouvait traverser la frontière GOOGL→SPY ($\sim 1000\$ \rightarrow \sim 300\$$) : un faux krach de -70% que l'`EvalCallback`

interprétait comme de la vraie donnée — la sélection du meilleur modèle était corrompue (le bug décisif de l’itération 4 du rapport). Deuxième piège, symétrique, découvert au chantier diffusion : la colonne `log_returns` du DataFrame final est *scalée* — le générateur recalcule ses rendements depuis la colonne `price` brute.

3 L’environnement (`environment.py`)

3.1 Espaces

Observation (52 dimensions) : fenêtre glissante de 10 jours $\times$ 5 features, aplatie, plus deux variables de portefeuille — position courante ($-1/0/1$) et PnL latent. **Actions** (discrètes, 4) : 0=Hold, 1=Long 100 %, 2=Flat, 3=Short 100 %. La sémantique est celle d’une machine à états : chaque action *cible* un état de position, et l’environnement exécute les transactions nécessaires (une transition short $\rightarrow$ long coûte deux trades : couverture puis achat).

3.2 Comptabilité — la mécanique du short

Toute la comptabilité tient dans deux variables, `_cash` et `_shares` (négatif = dette de titres), et une identité :

$$V_t = \text{cash} + \text{shares} \times P_t.$$

Ouvrir un short de N titres au prix P_0 : `_shares` $= -N$, `_cash` $+= NP_0$ (proceeds), d’où $V_t = V_0 + N(P_0 - P_t)$ — profit si le prix baisse, exactement la définition. Les frais (0.1 % par trade) sont débités du cash à *chaque* passage, y compris à l’ouverture du short (le bug historique n°1 du projet : frais d’ouverture du short non débités, trouvé par le premier test de comptabilité).

3.3 Récompense : l’alpha par pas de temps

```
portfolio_return = (V_t - V_prev) / V_prev
bh_return        = (P_t - P_prev) / P_prev
alpha            = portfolio_return - bh_return
reward          = clip(alpha * 100, -5, +5) # reward_scaling, borne PPO
```

Etre flat pendant une baisse $\rightarrow$ récompense ; flat pendant une hausse $\rightarrow$ pénalité ; long $\rightarrow$ neutre. L’agent est payé pour *timer* le marché, pas pour le porter (le rapport, §« alpha de Jensen », en donne la lecture financière).

3.4 Fins d’épisode et modes de départ

Deux sorties distinctes au sens Gymnasium : `terminated` = kill-switch (drawdown $> 25\%$ depuis le pic — une *règle du jeu*, pas une prédiction), `truncated` = fin du segment de données. Au `reset`, deux modes : départ aléatoire dans un segment aléatoire (entraînement — la régularisation par les données), ou `options={"random_start": False}` pour l’épisode déterministe full-split de l’évaluation.

4 Entraînement RL (`train.py`)

PPO (Stable-Baselines3), 4 environnements parallèles (`DummyVecEnv`) enveloppés dans `VecNormalize(norm_obs=True, norm_reward=True)` : les observations *et* les récompenses sont standardisées en ligne pendant l’entraînement. Hyperparamètres non par défaut, et pourquoi :

<code>clip_range</code>	0.15	updates de politique plus conservateurs que le 0.2 standard
<code>ent_coef</code>	0.05	force l'exploration des 4 actions (sinon l'agent gèle sur Hold/Long)
<code>n_steps/batch</code>	1024/128	$4 \times 1024 = 4096$ transitions par update
<code>total_timesteps</code>	800k	~10–15 min CPU
<code>gamma</code>	0.99	horizon effectif ~100 jours

La sélection de modèle est un *early stopping par la validation* : un `EvalCallback` évalue périodiquement sur le split val et conserve `best_model.zip` — en normalisant les observations d'évaluation avec les statistiques du `VecNormalize` d'entraînement (cohérence train/éval). Le `vec_normalize.pkl` est sauvé à côté du modèle : les deux forment un artefact indivisible.

Piège vécu. Prédire sur observations brutes avec un modèle entraîné sur observations normalisées donne des résultats faux mais plausibles — l'erreur a été commise deux fois dans l'histoire du projet. D'où le contrat unique : tout chargement passe par `evaluate.load_model_and_norm()`, qui recharge le `.pkl` adjacent, fige ses statistiques (`training=False`) et désactive la normalisation de récompense.

5 Évaluation (`evaluate.py` et scripts de robustesse)

- **Full-split déterministe** : départ fixe, politique `deterministic=True`, tout le split de test. Si le kill-switch stoppe l'épisode, la stratégie est *gelée en cash jusqu'à la fin de la fenêtre* (`freeze_after_stop`) — sinon les horizons ne seraient pas comparables entre stratégies stoppées et non stoppées.
- **Garde-fou de config** : `EVAL_ENV_CFG` doit rester alignée sur la config d'entraînement (frais 0.001, fenêtre 10, drawdown 25 %) — évaluer avec d'autres frais que ceux appris fausse tout.
- **Robustesse** : 5 sous-fenêtres aléatoires du test ; **cross-ticker** : le modèle Multi évalué sur le test de chaque actif.
- `walk_forward.py` : 5 réentraînements à fenêtre croissante ancrée en 2010, une année civile 100 % out-of-sample par fold (2018→2022), grille année×actif → `reports/walk_forward.json`.
- `seed_robustness.py` : 4 réentraînements du Multi à seeds différents → la *bande de bruit inter-seeds*, le juge de paix de toute expérience ultérieure (convention du repo : un candidat n'est promu que s'il gagne *hors* de cette bande).

6 Le générateur de scénarios (package `diffusion/`)

6.1 Données (`dataset.py`)

Fenêtres glissantes de $L = 256$ jours de log-rendements *bruts* (recalculés depuis `price`), stride 1, confinées par `segment_id` → 10 015 fenêtres sur le split train RL uniquement (zéro fuite vers val/test : condition pour brancher un jour la Phase 2). Normalisation *globale* — un seul couple (μ, σ) poolé, persisté dans `norm.json` : normaliser par fenêtre écraserait les régimes de volatilité, qui sont précisément ce qu'on veut apprendre. (C'est aussi, on le sait depuis le verdict, ce qui rend le problème dur : le modèle doit porter un *mélange* d'échelles.)

6.2 Processus de diffusion (`schedule.py`)

Tout le forward process tient dans les $\bar{\alpha}_t$ (produits cumulés des $1 - \beta_t$, schedule cosinus) et une forme fermée — pas besoin d'itérer les T pas pendant l'entraînement :

```
def q_sample(self, x0, t, noise):
    """x_t = sqrt(abar_t) * x0 + sqrt(1 - abar_t) * eps"""
    ab = self.alpha_bars[t].view(-1, 1, 1)
    return ab.sqrt() * x0 + (1.0 - ab).sqrt() * noise
```

6.3 Réseau (model.py) et objectifs (ddpm.py)

U-Net 1D de 0.56M paramètres : trois résolutions (256/128/64), blocs résiduels `Conv1d + GroupNorm + SiLU`, skip-connections symétriques, timestep injecté par embedding sinusoïdal + MLP dans chaque bloc. Conv finale initialisée à zéro (le réseau démarre en prédisant 0 : premiers pas d'entraînement stables). Deux objectifs implémentés : ε -prediction (v1) et **v-prediction** (v2), $v = \sqrt{\bar{\alpha}} \varepsilon - \sqrt{1 - \bar{\alpha}} x_0$. Le sampling ancestral est paramétré par $\hat{x}_0$ avec clipping ($\pm 15\sigma$, garde-fou anti-divergence), et l'**EMA des poids** (decay 0.999) est le modèle évalué.

6.4 Le protocole de validation (metrics.py, baselines.py, discriminative.py)

Chaque composant encode une leçon apprise pendant la calibration (É1 a rejeté le protocole v1 quatre fois avant de juger le moindre modèle) :

- **ACF par fenêtre puis moyennée** (jamais sur les fenêtres concaténées : artefacts de jonction) ; bandes d'acceptation par *tirages de runs contigus* de fenêtres — des tirages i.i.d. de fenêtres chevauchantes (stride 1) écrasent la variance d'échantillonnage et rejettent à tort (le bootstrap, marginales exactes par construction, était rejeté).
- **Anti-mémorisation** : distance L2 au plus proche voisin réel, référencée contre la distance NN *leave-one-out du réel sur lui-même* en excluant les voisins chevauchants ($\pm L$ jours, même segment) — sans cette exclusion, le NN d'une fenêtre est sa voisine décalée d'un jour et la métrique ne mesure rien.
- **Juge discriminatif figé** : GRU(2→64) sur les canaux $(r, |r|)$ — donner $|r|$ en entrée rend le juge directement sensible à la dynamique de volatilité — sorties moyennées dans le temps, entrées clippées à $\pm 8\sigma$ (anti-collapse), médiane de 3 seeds (le juge mono-seed était bimodal : 0.88/0.94/0.50). Split train/test par *blocs contigus tirés au hasard avec embargo* (purged split, de Prado) : un split aléatoire fuit par le chevauchement, un split « fin de période en test » fait apprendre l'époque au lieu du réalisme.
- **Baselines à rôle** : gaussienne i.i.d. (doit échouer queues et clustering), bootstrap i.i.d. (marginales parfaites, clustering détruit), GARCH(1,1)- t fitté par ticker via `arch` mais *simulé en numpy maison* (déterminisme contrôlé par notre générateur — `arch.simulate` dépend d'un état global).
- **Pré-enregistrement mécanique** : les bandes et ancres vivent dans `reports/diffusion_validation.json` le script les *réutilise* si la config est compatible au lieu de les recalculer — le contrat « seuils écrits avant l'entraînement » est appliqué par le code, pas par la bonne volonté.

7 La philosophie de tests — et le test le plus rentable du projet

`conftest.py` ne fournit *que* des données synthétiques : GBM pour les propriétés statistiques, prix constants ou à taux fixe pour la comptabilité *exacte* des trades (on peut asserter le cash au centime). Le protocole de validation est lui-même testé sur des processus aux propriétés connues : ACF analytique d'un AR(1) ($\rho_k = \varphi^k$), clustering d'un GARCH simulé à la main, kurtosis d'une Student. Le test le plus rentable du projet — écrit quand v1 et v2 ont échoué de façon identique et qu'il fallait départager « modèle mauvais » de « sampler buggé » : remplacer le réseau par le débruiteur *optimal analytique* pour des données $\mathcal{N}(0, 1)$ et vérifier que la boucle de génération reproduit $\mathcal{N}(0, 1)$:

```

class Oracle:
    # E[eps | x_t] en forme fermee
    def __call__(self, x, t):
        if pred_type == "eps":
            return (1 - sched.alpha_bars[t[0]]).sqrt() * x
        return torch.zeros_like(x) # E[v | x_t] = 0 pour N(0,1)

out = DDPM.sample(ddpm_avec_oracle, 300, 64, seed=3)
assert abs(out.std() - 1.0) < 0.03 # sampler exact => les verdicts tiennent

```

Verdict : std = 0.9985 pour les deux objectifs — le sampler est exact, le NO-GO est un fait du *modèle* (sous-engagement d'échelle sur le mélange multi-tickers), pas un bug. Quinze lignes qui ont sauvé la validité de deux runs de plusieurs heures.

8 Récapitulatif des pièges qui ont coûté cher

Piège	Symptôme	Garde-fou actuel
Obs brutes au predict (2×)	résultats faux mais plausibles	load_model_and_norm() obligatoire
Épisodes trans-tickers en val	faux krachs −70 %, best model tiré au sort	segment_id sur les 3 splits + test
Frais du short non débités	alpha gonflé silencieusement	tests de comptabilité exacte
Colonne log_returns scalée	distribution générative fausse	recalcul depuis price + test
Bandes i.i.d. sur fenêtres chevau- chantes	bootstrap rejeté à tort (É1)	tirages par runs contigus
Split asymétrique du juge	sanité 0.39 au lieu de 0.5	purged split + embargo, sy- métrique
$\hat{\nu} < 4$ au MLE GARCH- t	kurtosis échantillon 45 (« gold standard » hors bande)	É2 du GARCH reporté, mar- ginales jugées par le bootstrap
Divergence du sampling non en- traîné	valeurs $\sim 10^3$	clip de $\hat{x}_0$ + test-oracle

9 Conventions pour étendre le projet

- Un candidat s'entraîne dans SON dossier (models/<exp>/) et n'est promu que s'il bat la bande de bruit inter-seeds (RL) ou passe É1–É6 (générateur).
- PREDICTION.md écrit dans le dossier d'expérience *avant* chaque run, confronté au mesuré dans le rapport.
- Toute nouvelle métrique du protocole doit d'abord être testée sur une distribution dont la réponse est connue.
- Les bandes de diffusion_validation.json sont figées — juger un nouveau générateur : python validate_diffusion.py --model-dir models/<exp>/, jamais de recalibration sans nouvelle question de rapport.